GUIDE PRATIQUE · ADMINISTRATION LINUX

20 commandes DNF indispensables

Installer, mettre à jour, annuler et entretenir les paquets sur Fedora, RHEL, Rocky Linux et AlmaLinux

Installation, recherche, historique et retour arrière, dépôts, groupes, modules et maintenance : 20 fiches pratiques aux commandes vérifiées, un chapitre DNF5 et une routine sûre pour la production.

```
terminal
$ sudo dnf install -y nginx
$ dnf check-update --security
$ sudo dnf upgrade --refresh
$ sudo dnf history undo last
$ sudo dnf autoremove && sudo dnf clean all
```

RÉALISÉ PAR

Ngouye GNING

VERSION
1.0

DATE
Octobre 2026

CONTENU
20 fiches · 5 parties · DNF5

NAVIGATION

Sommaire

Introduction 3

DNF en un coup d'œil 5

Installer et mettre à jour 8
- 01 dnf install — Installer un paquet et ses dépendances 9
- 02 dnf remove — Désinstaller un paquet proprement 10
- 03 dnf check-update — Vérifier les mises à jour disponibles 11
- 04 dnf upgrade — Mettre à jour un paquet ou tout le système 12
- 05 dnf install ./x.rpm — Installer un RPM local ou distant 13

Rechercher et inspecter 14
- 06 dnf search — Rechercher un paquet par mot-clé 15
- 07 dnf info — Afficher le détail d'un paquet 16
- 08 dnf list — Lister les paquets selon leur état 17
- 09 dnf provides — Trouver le paquet qui fournit un fichier 18

Historique et retour arrière 19
- 10 dnf history — Consulter l'historique des transactions 20
- 11 dnf history undo — Annuler ou rejouer une transaction 21
- 12 dnf downgrade — Revenir à une version antérieure 22
- 13 dnf reinstall — Réinstaller un paquet endommagé 23

Dépôts, groupes et modules 24
- 14 dnf repolist — Lister et activer des dépôts 25
- 15 dnf config-manager — Ajouter et configurer des dépôts 26
- 16 dnf group — Installer un ensemble de paquets 27
- 17 dnf module — Choisir la version d'une application 28

Entretien et vérification 29
- 18 dnf autoremove — Supprimer les dépendances orphelines 30
- 19 dnf clean — Vider et reconstruire le cache 31
- 20 dnf check — Vérifier la cohérence de la base RPM 32

DNF5, la nouvelle génération 33
- Les commandes en pratique 33
- Ce qui change au quotidien 34
- Bon à savoir avant de migrer 34

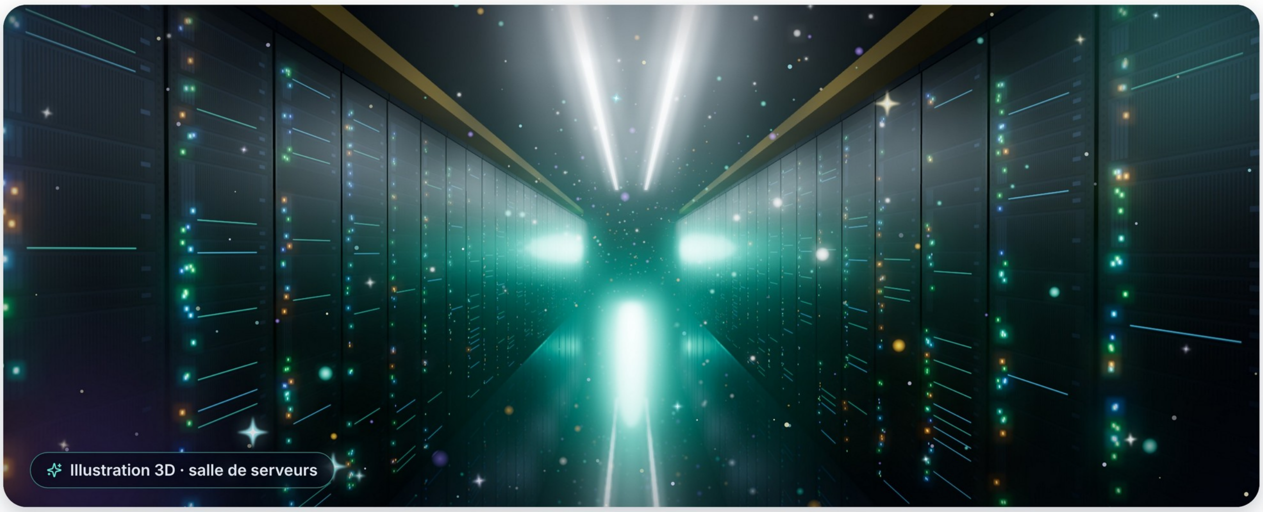
Conclusion 35

Annexes 36
- A. Aide-mémoire : « Je veux… » 36
- B. Équivalences apt ↔ dnf 37
- C. Fichiers et répertoires de DNF 37
- D. Réglages utiles de dnf.conf 38
- E. Messages d'erreur courants 39
- F. Glossaire 39
- G. Sources et références 40

AVANT-PROPOS

Introduction

Sur Fedora, Red Hat Enterprise Linux, Rocky Linux, AlmaLinux ou CentOS Stream, presque toute la vie d'un serveur passe par une seule commande : `dnf`. Installer un service, appliquer les correctifs de sécurité, revenir en arrière après une mise à jour problématique, activer un dépôt ou libérer de l'espace disque : tout commence par elle.



DNF (*Dandified YUM*) a succédé à YUM comme gestionnaire de paquets de la famille Red Hat. Il s'appuie sur RPM pour installer les fichiers, mais ajoute ce que RPM ne fait pas seul : la recherche dans les dépôts, la **résolution automatique des dépendances**, la vérification des signatures et un **historique des transactions** qui permet d'annuler une opération.

> **DES COMMANDES QUI NE SE TRADUISENT PAS**
> La traduction automatique de l'article d'origine a francisé les commandes : « dnf installer », « dnf supprimer », « liste de groupes dnf », « vérification sudo dnf »… Aucune de ces formes ne fonctionne. Les commandes DNF s'écrivent toujours en anglais : ce guide les rétablit toutes, telles que vous devez les taper (`dnf install`, `dnf remove`, `dnf group list`, `sudo dnf check`…).

Objectifs du guide

- Maîtriser les **20 commandes DNF** du quotidien, de l'installation à la vérification de la base RPM ;
- Savoir **annuler** une transaction, revenir à une version antérieure ou réparer un paquet endommagé ;
- Gérer **dépôts, groupes et modules** sans fragiliser le système ;
- Anticiper la transition vers **DNF5**, le gestionnaire par défaut de Fedora depuis la version 41.

Public et environnement

Ce guide s'adresse aux administrateurs systèmes, ingénieurs DevOps et candidats aux certifications RHCSA ou RHCE. Les exemples sont rédigés pour DNF 4, utilisé par RHEL 8, 9 et 10, Rocky Linux, AlmaLinux et CentOS Stream. Les différences avec DNF5 (Fedora 41 et suivantes) sont signalées au fil des fiches et détaillées dans un chapitre dédié.

Conventions utilisées

NOTATION	SIGNIFICATION
`$ sudo commande`	Commande qui modifie le système : privilèges administrateur (root) requis
`$ commande`	Commande de consultation, utilisable sans privilège
`nginx`, `curl`	Paquets d'exemple : remplacez-les par les vôtres
`# texte`	Commentaire explicatif dans un bloc de code
Essentiel **Pratique** **Avancé**	Niveau de chaque fiche : usage quotidien, régulier ou ponctuel

> **À PROPOS DES EXEMPLES**
>
> Les sorties sont illustratives : versions, tailles, numéros de transaction et noms de dépôts varient selon la distribution et la date. Elles sont reproduites en anglais, telles que DNF les affiche. La plupart ont été prises sur Rocky Linux 9.

SYNTHÈSE

DNF en un coup d’œil

Syntaxe : `dnf [options] commande [arguments]`. À chaque appel, DNF lit sa configuration, met à jour si besoin les métadonnées des dépôts, calcule une **transaction** cohérente, puis confie l’installation à RPM — avant d’enregistrer le tout dans son historique.

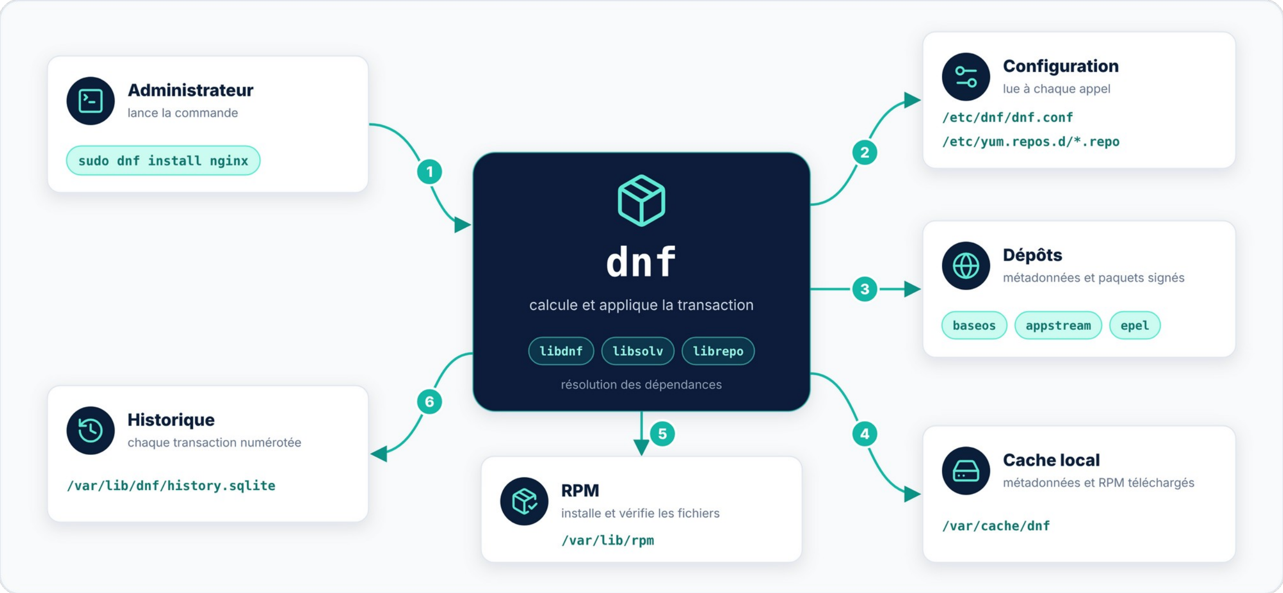

Administrateur
lance la commande
sudo dnf install nginx
1
dnf
calcule et applique la transaction
libdnf
libsolv
librepo
résolution des dépendances
2
Configuration
lue à chaque appel
/etc/dnf/dnf.conf
/etc/yum.repos.d/*.repo
3
Dépôts
métadonnées et paquets signés
baseos
appstream
epel
4
Cache local
métadonnées et RPM téléchargés
/var/cache/dnf
5
RPM
installe et vérifie les fichiers
/var/lib/rpm
6
Historique
chaque transaction numérotée
/var/lib/dnf/history.sqlite

Figure 1 — *Le cheminement d’une commande DNF, de la configuration à l’historique.*

> **DNF OU RPM ?**
>
> `rpm` installe, interroge et vérifie des paquets **un par un**, sans jamais aller chercher de dépendance. `dnf` travaille au niveau des **dépôts** : il trouve, télécharge et installe tout ce qui est nécessaire, en une transaction. Règle pratique : `dnf` pour installer et mettre à jour, `rpm -q…` et `rpm -V` pour interroger et vérifier.

Les 20 commandes du guide

N°	COMMANDE	RÔLE	NIVEAU
01	`dnf install`	Installer un paquet et ses dépendances	Essentiel
02	`dnf remove`	Désinstaller un paquet proprement	Essentiel
03	`dnf check-update`	Lister les mises à jour disponibles (code 100)	Essentiel
04	`dnf upgrade`	Mettre à jour un paquet ou tout le système	Essentiel
05	`dnf install ./x.rpm`	Installer un RPM local avec ses dépendances	Pratique
06	`dnf search`	Rechercher par nom et résumé (`--all` : description)	Essentiel
07	`dnf info`	Afficher le détail d’un paquet	Essentiel
08	`dnf list`	Lister les paquets installés, disponibles, à mettre à jour	Essentiel
09	`dnf provides`	Trouver le paquet qui fournit un fichier	Pratique
10	`dnf history`	Consulter le journal des transactions	Pratique
11	`dnf history undo`	Annuler ou rejouer une transaction	Avancé
12	`dnf downgrade`	Revenir à une version antérieure	Avancé
13	`dnf reinstall`	Réinstaller un paquet endommagé	Pratique
14	`dnf repolist`	Lister les dépôts, les activer ponctuellement	Essentiel
15	`dnf config-manager`	Ajouter et configurer des dépôts	Pratique
16	`dnf group`	Installer un ensemble cohérent de paquets	Pratique
17	`dnf module`	Choisir la version d’une application (RHEL 8 et 9)	Avancé
18	`dnf autoremove`	Supprimer les dépendances devenues inutiles	Pratique
19	`dnf clean`	Vider et reconstruire le cache	Pratique
20	`dnf check`	Vérifier la cohérence de la base RPM	Avancé

Essentiel dès le premier jour **Pratique** usage régulier **Avancé** dépannage et cas particuliers

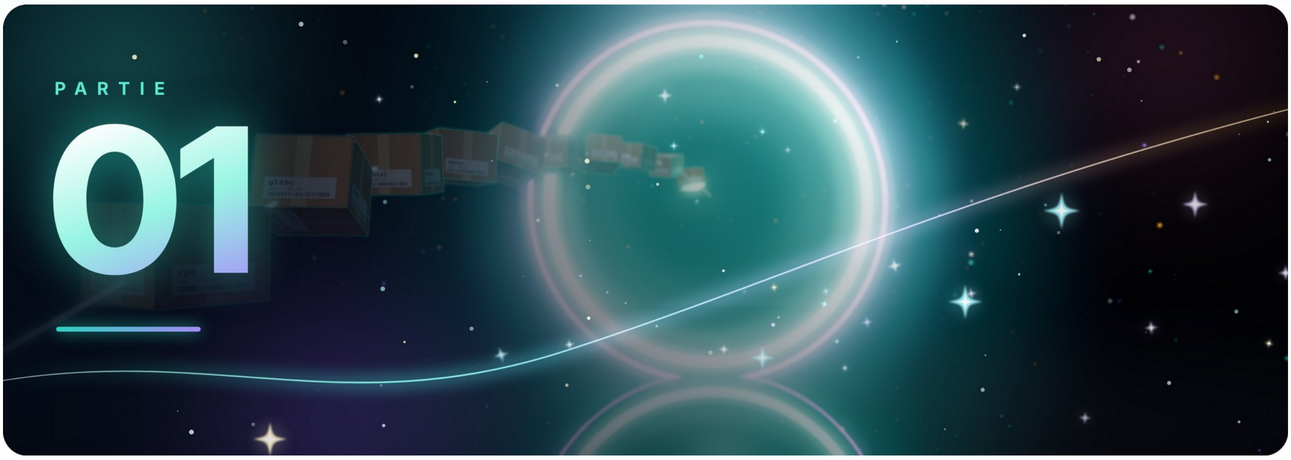
Les options globales à connaître

Ces options se placent avant ou après la commande et fonctionnent avec la plupart d'entre elles :

OPTION	EFFET
`-y, --assumeyes`	Répondre « oui » à toutes les questions (scripts, automatisation)
`--assumeno`	Répondre « non » : affiche la transaction sans l'exécuter
`-q, --quiet`	Sortie réduite à l'essentiel, pratique dans les scripts
`--refresh`	Forcer le rafraîchissement des métadonnées avant d'agir
`--enablerepo / --disablerepo`	Activer ou désactiver un dépôt pour cette seule commande
`-x, --exclude=motif`	Exclure des paquets de la transaction
`--setopt=option=valeur`	Modifier un réglage de dnf.conf pour cette seule commande
`--downloadonly`	Télécharger les paquets sans les installer
`--allowerasing`	Autoriser la suppression de paquets en conflit (à manier avec soin)

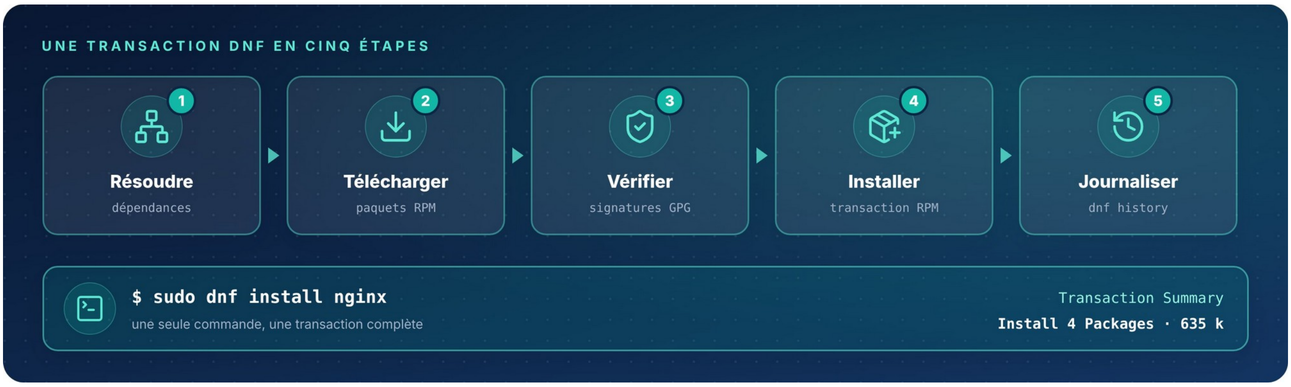
SIMULER AVANT D'AGIR

`sudo dnf upgrade --assumeno` affiche exactement ce que ferait la mise à jour — paquets, versions, tailles — puis s'arrête sans rien modifier. C'est le moyen le plus simple de préparer une fenêtre de maintenance.



Installer et mettre à jour

Installer un service, le retirer proprement, vérifier puis appliquer les mises à jour : ce sont les gestes que vous répéterez le plus souvent. À chaque fois, DNF calcule une transaction complète et vous la présente avant d'agir.


UNE TRANSACTION DNF EN CINQ ÉTAPES

1
Résoudre
dépendances
2
Télécharger
paquets RPM
3
Vérifier
signatures GPG
4
Installer
transaction RPM
5
Journaliser
dnf history

```
$ sudo dnf install nginx
```

une seule commande, une transaction complète

```
Transaction Summary
Install 4 Packages · 635 k
```

DANS CETTE PARTIE

01	`dnf install`	Installer un paquet et ses dépendances
02	`dnf remove`	Désinstaller un paquet proprement
03	`dnf check-update`	Vérifier les mises à jour disponibles
04	`dnf upgrade`	Mettre à jour un paquet ou tout le système
05	`dnf install ./x.rpm`	Installer un RPM local ou distant

FICHE 01 · INSTALLATION

01 `dnf install` — Installer un paquet et ses dépendances

OBJECTIF	PRIVILÈGES	NIVEAU
Ajouter un logiciel depuis les dépôts	root (sudo)	Essentiel

`dnf install` télécharge le paquet demandé **et toutes ses dépendances**, vérifie leurs signatures GPG, puis les installe en une seule transaction. Avant d'agir, DNF affiche le résumé de la transaction et attend votre confirmation.

```bash
$ sudo dnf install nginx
Dependencies resolved.
================================================================================
 Package               Arch      Version                Repository        Size
================================================================================
Installing:
 nginx                 x86_64    1:1.20.1-22.el9        appstream         36 k
Installing dependencies:
 nginx-core            x86_64    1:1.20.1-22.el9        appstream        566 k
 nginx-filesystem      noarch    1:1.20.1-22.el9        appstream        8.4 k
 rocky-logos-httpd     noarch    90.16-1.el9            appstream         24 k

Transaction Summary
================================================================================
Install  4 Packages

Total download size: 635 k
Installed size: 1.8 M
Is this ok [y/N]: y
```

Plusieurs paquets peuvent être installés en une seule commande. L'option `-y` répond automatiquement « oui », ce qui est utile dans les scripts :

```bash
$ sudo dnf install -y nginx php-fpm
# Installer une version précise
$ sudo dnf install nginx-1:1.20.1-20.el9
# Installer à partir d'un fichier fourni, sans connaître le nom du paquet
$ sudo dnf install /usr/bin/dig
```

LIRE AVANT DE CONFIRMER

Le résumé de transaction est votre dernier contrôle. Une installation qui fait apparaître des sections « Removing » ou « Downgrading » mérite toujours une analyse avant de répondre `y`.

FICHE 02 · INSTALLATION

02 `dnf remove` — Désinstaller un paquet proprement

OBJECTIF	PRIVILÈGES	NIVEAU
Retirer un logiciel et ses dépendances inutiles	root (sudo)	**Essentiel**

`dnf remove` désinstalle le paquet indiqué **ainsi que les paquets qui en dépendent**. Avec le réglage par défaut `clean_requirements_on_remove=True`, DNF retire aussi les dépendances installées automatiquement et devenues inutiles.

```bash
$ sudo dnf remove nginx
Dependencies resolved.
================================================================================
 Package               Arch      Version                Repository        Size
================================================================================
Removing:
 nginx                 x86_64    1:1.20.1-22.el9        @appstream       113 k
Removing unused dependencies:
 nginx-core            x86_64    1:1.20.1-22.el9        @appstream       1.7 M
 nginx-filesystem      noarch    1:1.20.1-22.el9        @appstream       141
 rocky-logos-httpd     noarch    90.16-1.el9            @appstream        14 k

Transaction Summary
================================================================================
Remove  4 Packages

Freed space: 1.8 M
Is this ok [y/N]:
```

EFFET DOMINO

Si d’autres paquets dépendent de celui que vous retirez, ils figurent dans la section « Removing dependent packages » et seront supprimés avec lui. Retirer une bibliothèque peut ainsi emporter une grande partie du système : en cas de doute, répondez `N`.

Certains paquets vitaux — `dnf`, `systemd`, `sudo`, le noyau en cours d’exécution — sont **protégés** par les fichiers de `/etc/dnf/protected.d/` : DNF refuse de les supprimer.

```bash
$ sudo dnf remove sudo
Error:
 Problem: The operation would result in removing the following protected
 packages: sudo
```

FICHE 03 · INSTALLATION

03 `dnf check-update` — Vérifier les mises à jour disponibles

OBJECTIF	PRIVILÈGES	NIVEAU
Savoir ce qui sera mis à jour, sans rien modifier	Aucun	**Essentiel**

`dnf check-update` rafraîchit les métadonnées si nécessaire et liste les paquets pour lesquels une version plus récente existe. Elle ne modifie rien : c'est la commande à lancer avant toute mise à jour en production.

```bash
$ dnf check-update
Last metadata expiration check: 0:03:12 ago on Tue 01 Sep 2026 10:02:11 AM UTC.

kernel.x86_64                   5.14.0-570.42.1.el9_6         baseos
openssl.x86_64                  1:3.2.2-6.el9_6.1             baseos
openssl-libs.x86_64             1:3.2.2-6.el9_6.1             baseos
$ echo $?
100
```

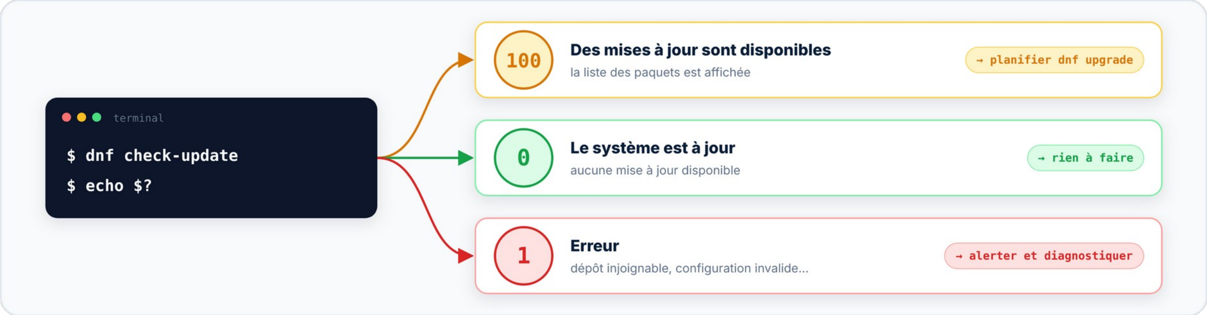

terminal
$ dnf check-update
$ echo $?
100
Des mises à jour sont disponibles
la liste des paquets est affichée
→ planifier dnf upgrade
0
Le système est à jour
aucune mise à jour disponible
→ rien à faire
1
Erreur
dépôt injoignable, configuration invalide...
→ alerter et diagnostiquer

Figure 2 — *Le code de retour de dnf check-update, exploitable dans les scripts de supervision.*

```bash
#!/bin/bash
dnf -q check-update > /tmp/maj.txt
case $? in
  100) echo "Mises à jour disponibles : $(grep -c . /tmp/maj.txt)" ;;
  0)   echo "Système à jour" ;;
  *)   echo "Erreur lors de la vérification" >&2 ; exit 1 ;;
esac
```

> **CORRECTIFS DE SÉCURITÉ SEULEMENT**
>
> Sur RHEL et ses dérivés, `dnf check-update --security` ne liste que les correctifs de sécurité, grâce aux avis publiés par la distribution (*updateinfo*) ; `dnf updateinfo list --security` les détaille, avis par avis.

FICHE 04 · INSTALLATION

04 `dnf upgrade` — Mettre à jour un paquet ou tout le système

OBJECTIF	PRIVILÈGES	NIVEAU
Appliquer correctifs et nouvelles versions	root (sudo)	**Essentiel**

`dnf upgrade` met à jour tous les paquets du système, ou seulement ceux que vous indiquez. `dnf update` en est un alias historique : les deux commandes sont équivalentes, mais `upgrade` est la forme recommandée.

```bash
# Tout le système, en forçant le rafraîchissement des métadonnées
$ sudo dnf upgrade --refresh
# Un seul paquet
$ sudo dnf upgrade openssl
# Uniquement les correctifs de sécurité (RHEL, Rocky Linux, AlmaLinux)
$ sudo dnf upgrade --security
# Tout mettre à jour, sauf le noyau
$ sudo dnf upgrade --exclude='kernel*'
```

Faut-il redémarrer ?

Un noyau, la glibc ou systemd mis à jour ne sont pris en compte qu'après un redémarrage. La commande `needs-restarting`, fournie par `dnf-plugins-core`, répond à la question :

```bash
$ sudo dnf needs-restarting -r
Core libraries or services have been updated since boot-up:
  * kernel
  * systemd

Reboot is required to fully utilize these updates.
More information: https://access.redhat.com/solutions/27943
```

> **CODE DE RETOUR**
>
> `needs-restarting -r` renvoie 1 lorsqu'un redémarrage est nécessaire et 0 sinon : idéal dans un playbook. Sans `-r`, la commande liste les processus qui utilisent encore d'anciennes bibliothèques et qu'il suffit de relancer.

FICHE 05 · INSTALLATION

05 `dnf install ./x.rpm` — Installer un RPM local ou distant

OBJECTIF	PRIVILÈGES	NIVEAU
Installer un fichier .rpm, dépendances comprises	root (sudo)	**Pratique**

Certains éditeurs distribuent leurs logiciels sous forme de fichier `.rpm`. Contrairement à `rpm -i`, qui échoue dès qu'une dépendance manque, `dnf install` va chercher les dépendances manquantes dans les dépôts configurés.

```bash
# Inspecter le fichier avant de l'installer
$ rpm -qpi ./app-2.4.1-1.x86_64.rpm          # éditeur, version, description
$ rpm -qpR ./app-2.4.1-1.x86_64.rpm          # dépendances requises
$ rpm -K ./app-2.4.1-1.x86_64.rpm            # signature et empreintes
./app-2.4.1-1.x86_64.rpm: digests signatures OK
# Installer : le « ./ » désigne un fichier local, pas un nom de paquet
$ sudo dnf install ./app-2.4.1-1.x86_64.rpm
# Installer directement depuis une URL
$ sudo dnf install \
      https://dl.fedoraproject.org/pub/epel/epel-release-latest-9.noarch.rpm
```

Le paquet installé est enregistré comme provenant de `@commandline` : il ne sera pas mis à jour par `dnf upgrade`, sauf si l'éditeur fournit aussi un dépôt.

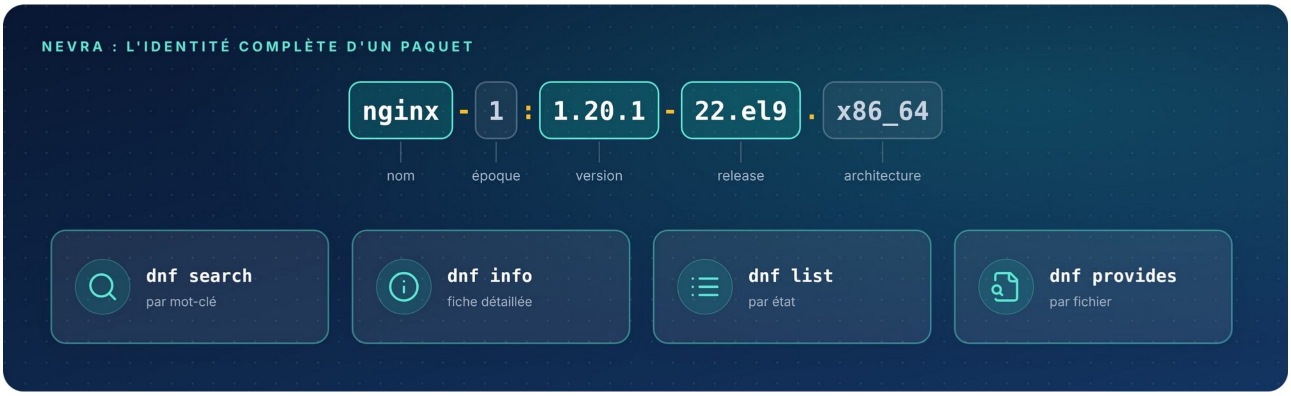
CRITÈRE	RPM -I FICHIER.RPM	DNF INSTALL ./FICHIER.RPM
Dépendances	Non résolues : l'installation échoue	Résolues depuis les dépôts actifs
Historique	Aucun	Transaction visible dans `dnf history`
Annulation	Manuelle (`rpm -e`)	`dnf history undo`
Signature	Contrôle manuel avec `rpm -K`	Non vérifiée par défaut (`localpkg_gpgcheck`)

SIGNATURE

Par défaut, DNF 4 ne vérifie pas la signature des fichiers locaux : d'où l'intérêt de `rpm -K`. S'il signale « SIGNATURES NOT OK » ou une clé manquante, importez la clé publique de l'éditeur (`sudo rpm --import URL_DE_LA_CLÉ`) après avoir vérifié sa provenance. N'utilisez pas `--nogpgcheck` pour « faire passer » un paquet : c'est précisément le contrôle qui vous protège d'un fichier modifié.

Rechercher et inspecter

Avant d’installer quoi que ce soit, il faut trouver le bon paquet : par mot-clé, par fichier fourni ou par état. Ces commandes ne modifient rien et ne demandent aucun privilège.


NEVRA : L'IDENTITÉ COMPLÈTE D'UN PAQUET
nginx - 1 : 1.20.1 - 22.el9 . x86_64
nom
époque
version
release
architecture
dnf search
par mot-clé
dnf info
fiche détaillée
dnf list
par état
dnf provides
par fichier

DANS CETTE PARTIE

06	**`dnf search`**	Rechercher un paquet par mot-clé
07	**`dnf info`**	Afficher le détail d’un paquet
08	**`dnf list`**	Lister les paquets selon leur état
09	**`dnf provides`**	Trouver le paquet qui fournit un fichier

FICHE 06 · RECHERCHE

06 `dnf search` — Rechercher un paquet par mot-clé

OBJECTIF	PRIVILÈGES	NIVEAU
Trouver le nom exact d'un paquet	Aucun	**Essentiel**

`dnf search` cherche un mot-clé dans le **nom** et le **résumé** des paquets disponibles, sans tenir compte de la casse. Les correspondances exactes sur le nom apparaissent en premier.

```bash
$ dnf search nginx
========================= Name Exactly Matched: nginx ==========================
nginx.x86_64 : A high performance web server and reverse proxy server
======================== Name & Summary Matched: nginx =========================
nginx-all-modules.noarch : A meta package that installs all available Nginx
                         : modules
nginx-core.x86_64 : nginx minimal core
nginx-filesystem.noarch : The basic directory layout for the Nginx server
nginx-mod-stream.x86_64 : Nginx stream modules
```

Ajoutez `--all` pour étendre la recherche à la **description** et à l'**URL** des paquets — utile quand on cherche une fonction plutôt qu'un nom :

```bash
$ dnf search --all "reverse proxy"
# Lister les paquets dont le nom suit un motif (jokers autorisés)
$ dnf list --available 'python3-*yaml*'
```

PAQUET INTROUVABLE ?

Un logiciel absent des résultats se trouve peut-être dans un dépôt désactivé : EPEL, CRB (CodeReady Builder) ou un dépôt d'éditeur. `dnf --enablerepo='*' search motclé` interroge tous les dépôts configurés, même désactivés — c'est plus lent, mais exhaustif.

FICHE 07 · RECHERCHE

07 `dnf info` — Afficher le détail d'un paquet

OBJECTIF	PRIVILÈGES	NIVEAU
Connaître version, dépôt, licence et description	Aucun	**Essentiel**

`dnf info` affiche la fiche d'identité d'un paquet : version, release, architecture, taille, dépôt, licence et description. Si le paquet est installé et qu'une mise à jour existe, les deux versions apparaissent.

```bash
$ dnf info nginx
Available Packages
Name         : nginx
Epoch        : 1
Version      : 1.20.1
Release      : 22.el9
Architecture : x86_64
Size         : 36 k
Source       : nginx-1.20.1-22.el9.src.rpm
Repository   : appstream
Summary      : A high performance web server and reverse proxy server
URL          : https://nginx.org
License      : BSD
Description  : Nginx is a web server and a reverse proxy server for HTTP, SMTP,
             : POP3 and IMAP protocols, with a strong focus on high concurrency,
             : performance and low memory usage.
```

Pour cibler uniquement la version installée — et savoir de quel dépôt elle provient — ajoutez `--installed` :

```bash
$ dnf info --installed nginx | grep -E 'Version|Release|Repository|From repo'
Version      : 1.20.1
Release      : 22.el9
Repository   : @System
From repo    : appstream
```

ALLER PLUS LOIN AVEC REPOQUERY

`dnf repoquery` interroge les métadonnées en détail : `dnf repoquery --requires nginx` liste ses dépendances, `dnf repoquery --whatrequires nginx` les paquets qui dépendent de lui, et `dnf repoquery -l nginx` les fichiers qu'il installe.

FICHE 08 · RECHERCHE

08 `dnf list` — Lister les paquets selon leur état

OBJECTIF	PRIVILÈGES	NIVEAU
Dresser un inventaire : installés, disponibles, à jour	Aucun	**Essentiel**

`dnf list` affiche les paquets selon leur état. Sans option, la commande liste à la fois les paquets installés et ceux disponibles dans les dépôts — souvent plusieurs milliers de lignes : filtrez toujours.

```bash
$ dnf list --installed | head -4
Installed Packages
NetworkManager.x86_64          1:1.52.0-5.el9_6            @baseos
acl.x86_64                     2.3.1-4.el9                 @anaconda
alternatives.x86_64            1.24-2.el9                  @baseos
$ dnf list --available 'php*'
$ dnf list --upgrades
# Compter les paquets installés (plus rapide avec rpm)
$ rpm -qa | wc -l
684
```

OPTION	PAQUETS AFFICHÉS
`--installed`	Installés (la colonne dépôt commence par @)
`--available`	Disponibles dans les dépôts et non installés
`--upgrades`	Installés pour lesquels une mise à jour existe
`--extras`	Installés mais absents de tous les dépôts actifs
`--recent`	Ajoutés récemment aux dépôts
`--autoremove`	Que `dnf autoremove` supprimerait
`--showduplicates`	Toutes les versions disponibles, pas seulement la plus récente

Quels paquets ont été installés en dernier ?

```bash
$ rpm -qa --last | head -3
nginx-1.20.1-22.el9.x86_64                        Fri 28 Aug 2026 02:02:35 PM UTC
nginx-core-1.20.1-22.el9.x86_64                   Fri 28 Aug 2026 02:02:34 PM UTC
rocky-logos-httpd-90.16-1.el9.noarch              Fri 28 Aug 2026 02:02:34 PM UTC
```

COMPATIBILITÉ DNF5

DNF 4 accepte encore les formes sans tirets (`dnf list installed`). DNF5 n'accepte que les options (`dnf list --installed`) : prenez dès maintenant cette habitude, valable partout.

FICHE 09 · RECHERCHE

09 `dnf provides` — Trouver le paquet qui fournit un fichier

OBJECTIF	PRIVILÈGES	NIVEAU
Retrouver un paquet à partir d'une commande ou d'une bibliothèque	Aucun	**Pratique**

Une commande introuvable, une bibliothèque manquante signalée par une application : `dnf provides` (alias `whatprovides`) indique quel paquet fournit un fichier ou une capacité donnée.

```bash
$ dnf provides /usr/sbin/semanage
policycoreutils-python-utils-3.6-2.1.el9.noarch : SELinux policy core python
                                                 : utilities
Repo        : appstream
Matched from:
Filename    : /usr/sbin/semanage

# Jokers : quand on ne connaît pas le chemin exact
$ dnf provides '*/bin/dig'
# Bibliothèque partagée réclamée par une application
$ dnf provides 'libssl.so.3()(64bit)'
```

Si vous indiquez seulement le nom d'une commande (`dnf provides semanage`), DNF essaie automatiquement les chemins courants, comme `/usr/bin` et `/usr/sbin`.

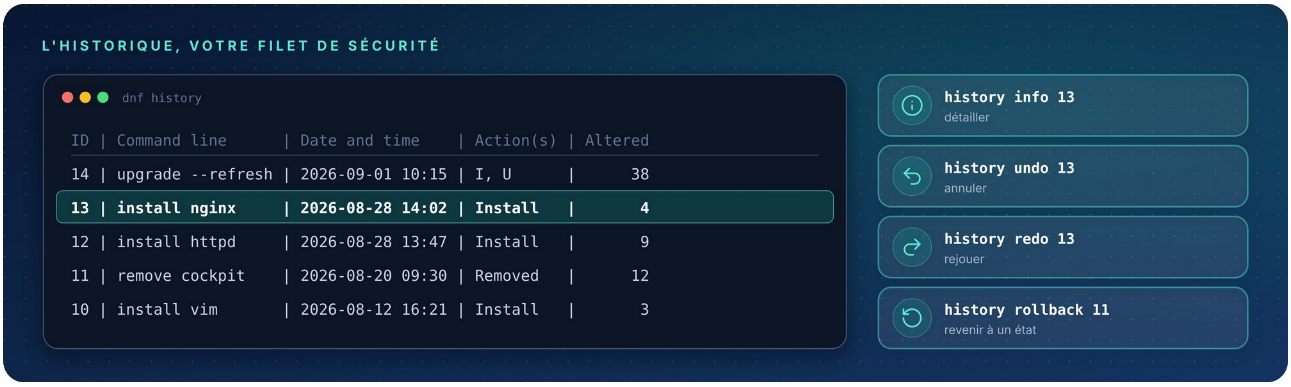
> **LES APOSTROPHES COMPTENT**
>
> Entourez toujours les jokers d'apostrophes (`'*/bin/dig'`) : sans elles, le shell tenterait de les développer lui-même avant de transmettre l'argument à DNF, avec un résultat imprévisible.

À l'inverse, pour savoir à quel paquet **installé** appartient un fichier déjà présent sur le disque, interrogez directement la base RPM :

```bash
$ rpm -qf /etc/ssh/sshd_config
openssh-server-8.7p1-45.el9.x86_64
```

Historique et retour arrière

Chaque opération DNF est enregistrée comme une transaction numérotée. Cet historique permet de savoir qui a changé quoi et quand, et surtout de revenir en arrière lorsqu’une mise à jour pose problème.



L'HISTORIQUE, VOTRE FILET DE SÉCURITÉ

```
dnf history

ID | Command line      | Date and time    | Action(s) | Altered
14 | upgrade --refresh | 2026-09-01 10:15 | I, U      |      38
13 | install nginx     | 2026-08-28 14:02 | Install   |       4
12 | install httpd     | 2026-08-28 13:47 | Install   |       9
11 | remove cockpit    | 2026-08-20 09:30 | Removed   |      12
10 | install vim       | 2026-08-12 16:21 | Install   |       3
```

`history info 13` — détailler

`history undo 13` — annuler

`history redo 13` — rejouer

`history rollback 11` — revenir à un état

DANS CETTE PARTIE

10	**`dnf history`**	Consulter l’historique des transactions
11	**`dnf history undo`**	Annuler ou rejouer une transaction
12	**`dnf downgrade`**	Revenir à une version antérieure
13	**`dnf reinstall`**	Réinstaller un paquet endommagé

FICHE 10 · HISTORIQUE

10 `dnf history` — Consulter l'historique des transactions

OBJECTIF	PRIVILÈGES	NIVEAU
Savoir qui a changé quoi, et quand	Aucun (lecture)	**Pratique**

Chaque opération qui modifie le système devient une **transaction numérotée**, enregistrée dans `/var/lib/dnf/history.sqlite`. `dnf history` (équivalent de `dnf history list`) affiche les plus récentes en premier.

```bash
$ dnf history
ID | Command line         | Date and time    | Action(s)      | Altered
-------------------------------------------------------------------------------
14 | upgrade --refresh    | 2026-09-01 10:15 | I, U           |       38
13 | install nginx        | 2026-08-28 14:02 | Install        |        4
12 | install httpd        | 2026-08-28 13:47 | Install        |        9
11 | remove cockpit       | 2026-08-20 09:30 | Removed        |       12
```

Les actions sont abrégées : **I** installation, **U** mise à jour, **E** suppression, **D** retour à une version antérieure, **R** réinstallation, **O** remplacement d'un paquet obsolète. Le détail d'une transaction s'obtient avec `history info` :

```bash
$ dnf history info 13
Transaction ID : 13
Begin time     : Fri 28 Aug 2026 02:02:31 PM UTC
End time       : Fri 28 Aug 2026 02:02:36 PM UTC (5 seconds)
User           : Alice Martin <alice>
Return-Code    : Success
Releasever     : 9
Command Line   : install nginx
Packages Altered:
    Install nginx-1:1.20.1-22.el9.x86_64                  @appstream
    Install nginx-core-1:1.20.1-22.el9.x86_64             @appstream
    Install nginx-filesystem-1:1.20.1-22.el9.noarch       @appstream
    Install rocky-logos-httpd-90.16-1.el9.noarch          @appstream
```

```bash
$ dnf history list nginx          # transactions qui concernent nginx
$ dnf history info last           # la dernière transaction
$ dnf history userinstalled       # paquets installés explicitement
```

FICHE 11 · HISTORIQUE

11 `dnf history undo` — Annuler ou rejouer une transaction

OBJECTIF	PRIVILÈGES	NIVEAU
Revenir sur une opération précise	root (sudo)	**Avancé**

`dnf history undo` applique l'**inverse** d'une transaction : ce qu'elle avait installé est supprimé, ce qu'elle avait supprimé est réinstallé, ce qu'elle avait mis à jour revient à la version précédente. `redo` rejoue la même transaction.

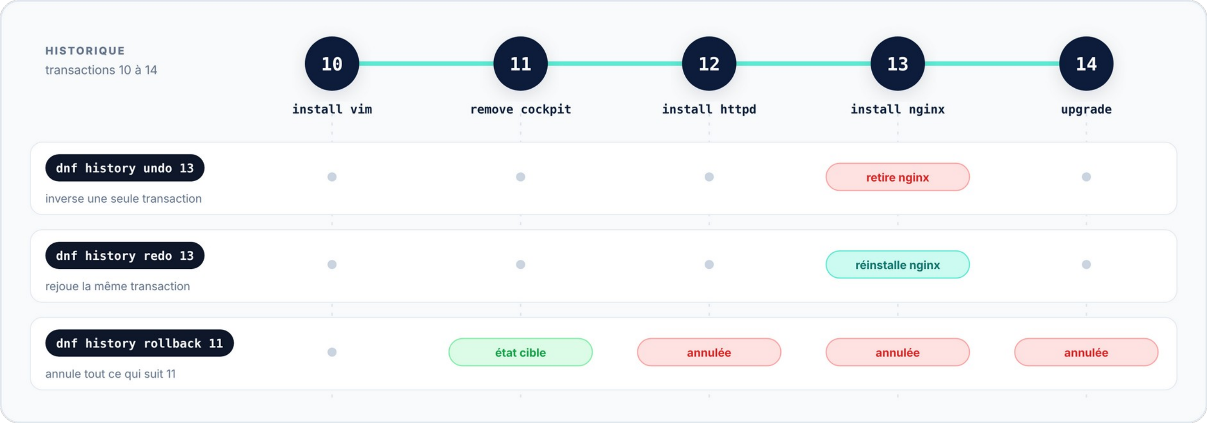

HISTORIQUE
transactions 10 à 14
10
install vim
11
remove cockpit
12
install httpd
13
install nginx
14
upgrade
dnf history undo 13
inverse une seule transaction
retire nginx
dnf history redo 13
rejoue la même transaction
réinstalle nginx
dnf history rollback 11
annule tout ce qui suit 11
état cible
annulée
annulée
annulée

Figure 3 — *undo annule une seule transaction ; rollback annule toutes celles qui suivent l'état choisi.*

```bash
# Annuler l'installation de nginx (transaction 13)
$ sudo dnf history undo 13
# La rejouer à l'identique
$ sudo dnf history redo 13
# Annuler la dernière transaction, quelle qu'elle soit
$ sudo dnf history undo last
# Revenir à l'état du système juste après la transaction 11
$ sudo dnf history rollback 11
```

LIMITES

Pour annuler une mise à jour, DNF doit retrouver l'**ancienne version** dans un dépôt ou dans le cache. Les dépôts de RHEL, Rocky Linux et AlmaLinux conservent les versions précédentes ; le dépôt *updates* de Fedora, non : l'annulation y échoue souvent. Quant à `rollback`, il annule toutes les transactions postérieures : à manier avec précaution.

FICHE 12 · HISTORIQUE

12 `dnf downgrade` — Revenir à une version antérieure

OBJECTIF	PRIVILÈGES	NIVEAU
Contourner une régression	root (sudo)	**Avancé**

Une mise à jour introduit une régression ? `dnf downgrade` installe une version plus ancienne d'un paquet — par défaut, la version immédiatement inférieure disponible.

```bash
# Quelles versions sont disponibles ?
$ dnf --showduplicates list nginx
Installed Packages
nginx.x86_64              1:1.20.1-22.el9               @appstream
Available Packages
nginx.x86_64              1:1.20.1-16.el9_4.1           appstream
nginx.x86_64              1:1.20.1-20.el9               appstream
nginx.x86_64              1:1.20.1-22.el9               appstream
# Version immédiatement inférieure
$ sudo dnf downgrade nginx
# Version précise
$ sudo dnf downgrade nginx-1:1.20.1-16.el9_4.1
```

Empêcher la prochaine mise à jour

Sans précaution, le prochain `dnf upgrade` réinstallera la version problématique. Le greffon *versionlock* fige un paquet à sa version actuelle :

```bash
$ sudo dnf install python3-dnf-plugin-versionlock
$ sudo dnf versionlock add nginx
$ dnf versionlock list
nginx-1:1.20.1-16.el9_4.1.*
# Lever le verrou une fois le correctif publié
$ sudo dnf versionlock delete nginx
```

> **ALTERNATIVE SANS GREFFON**
>
> La directive `exclude=nginx*` dans `/etc/dnf/dnf.conf` bloque aussi les mises à jour. Elle est plus brutale : le paquet devient invisible pour toutes les commandes, y compris l'installation.

FICHE 13 · HISTORIQUE

13 `dnf reinstall` — Réinstaller un paquet endommagé

OBJECTIF	PRIVILÈGES	NIVEAU
Restaurer des fichiers supprimés ou corrompus	root (sudo)	**Pratique**

Un binaire supprimé par erreur, une bibliothèque corrompue : `dnf reinstall` réinstalle la **même version** du paquet et restaure ses fichiers d'origine. Commencez par identifier le problème avec `rpm -V` :

```bash
$ sudo rm /usr/bin/curl                  # l'accident
$ rpm -V curl
missing     /usr/bin/curl
$ sudo dnf reinstall curl
$ rpm -V curl && echo "curl est intact"
curl est intact
```

La réinstallation exige que la version installée soit encore disponible dans un dépôt ou dans le cache. Si ce n'est plus le cas, mettez le paquet à jour (`dnf upgrade curl`) : le résultat est le même, les fichiers sont réécrits.

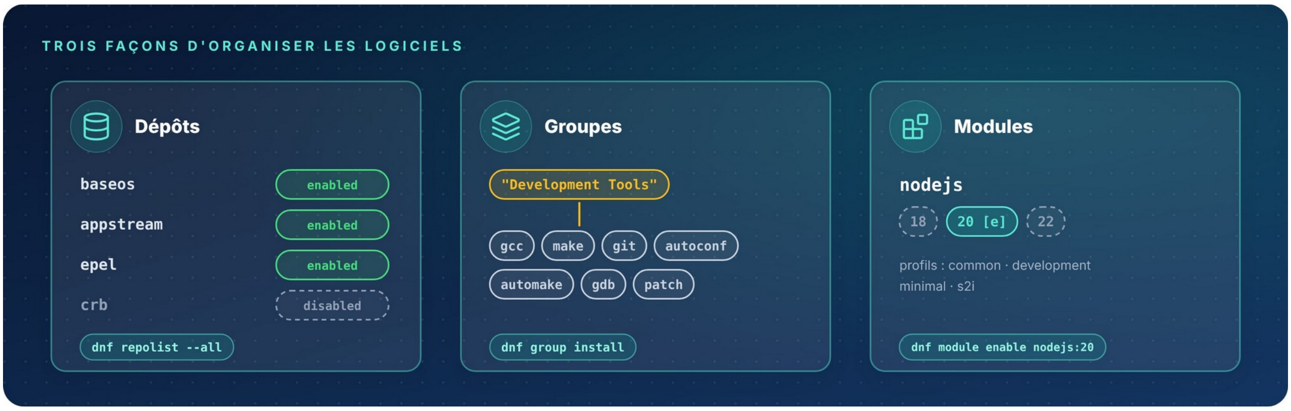
> **FICHIERS DE CONFIGURATION**
> Les fichiers de configuration que vous avez modifiés (marqués `%config(noreplace)` dans le paquet) sont conservés : `dnf reinstall` ne remet pas votre configuration à zéro.

Quelle commande pour quelle situation ?

SITUATION	COMMANDE
Binaire ou bibliothèque supprimé, corrompu	`sudo dnf reinstall paquet`
Version installée absente des dépôts	`sudo dnf upgrade paquet` ou `sudo dnf distro-sync paquet`
Retrouver un fichier de configuration d'origine	`dnf download paquet`, puis extraction avec `rpm2cpio`
Base RPM incohérente (doublons, dépendances)	`dnf check` (fiche 20)

Dépôts, groupes et modules

Les dépôts déterminent quels logiciels sont disponibles ; les groupes installent un ensemble cohérent de paquets en une commande ; les modules permettent de choisir la version d'une application sur RHEL 8 et 9.


TROIS FAÇONS D'ORGANISER LES LOGICIELS
Dépôts
baseos
enabled
appstream
enabled
epel
enabled
crb
disabled
Groupes
"Development Tools"
gcc
make
git
autoconf
automake
gdb
patch
Modules
nodejs
18
20 [e]
22
profils : common · development
minimal · s2i

```
dnf repolist --all
dnf group install
dnf module enable nodejs:20
```

DANS CETTE PARTIE

14	`dnf repolist`	Lister et activer des dépôts
15	`dnf config-manager`	Ajouter et configurer des dépôts
16	`dnf group`	Installer un ensemble de paquets
17	`dnf module`	Choisir la version d'une application

FICHE 14 · DÉPÔTS

14 `dnf repolist` — Lister et activer des dépôts

OBJECTIF	PRIVILÈGES	NIVEAU
Savoir d'où viennent les paquets	Aucun (lecture)	**Essentiel**

`dnf repolist` affiche les dépôts **activés** ; `--all` y ajoute les dépôts désactivés, avec leur état. Chaque dépôt est décrit par un fichier `.repo` dans `/etc/yum.repos.d/`.

```bash
$ dnf repolist
repo id                  repo name
appstream                Rocky Linux 9 - AppStream
baseos                   Rocky Linux 9 - BaseOS
epel                     Extra Packages for Enterprise Linux 9 - x86_64
extras                   Rocky Linux 9 - Extras
$ dnf repolist --all
repo id                  repo name                                        status
appstream                Rocky Linux 9 - AppStream                        enabled
baseos                   Rocky Linux 9 - BaseOS                           enabled
crb                      Rocky Linux 9 - CRB                             disabled
extras                   Rocky Linux 9 - Extras                           enabled
# Détails : URL, nombre de paquets, date de mise à jour
$ dnf repolist -v epel
```

Activer un dépôt le temps d'une commande

```bash
$ sudo dnf --enablerepo=crb install ninja-build
$ sudo dnf --disablerepo=epel upgrade
# N'interroger QUE le dépôt indiqué
$ dnf --repo=epel list --available | head
```

> **PONCTUEL OU PERSISTANT**
>
> `--enablerepo` et `--disablerepo` n'agissent que sur la commande en cours. Pour modifier durablement l'état d'un dépôt, utilisez `dnf config-manager` (fiche 15).

FICHE 15 · DÉPÔTS

15 `dnf config-manager` — Ajouter et configurer des dépôts

OBJECTIF	PRIVILÈGES	NIVEAU
Activer, désactiver ou ajouter un dépôt durablement	root (sudo)	**Pratique**

`dnf config-manager`, fourni par le paquet `dnf-plugins-core`, modifie les fichiers `.repo` à votre place, sans édition manuelle.

bash

```bash
$ sudo dnf install -y dnf-plugins-core
# Activer durablement le dépôt CRB (Rocky Linux, AlmaLinux 9)
$ sudo dnf config-manager --set-enabled crb
# Désactiver un dépôt
$ sudo dnf config-manager --set-disabled epel-testing
# Ajouter le dépôt d'un éditeur à partir de son fichier .repo
$ sudo dnf config-manager \
      --add-repo https://download.docker.com/linux/rhel/docker-ce.repo
```

DISTRIBUTION	ACTIVER LES PAQUETS DE DÉVELOPPEMENT
Rocky Linux, AlmaLinux 9 et 10	`sudo dnf config-manager --set-enabled crb`
Rocky Linux, AlmaLinux 8	`sudo dnf config-manager --set-enabled powertools`
RHEL 9 (abonnement)	Dépôt `codeready-builder-for-rhel-9-$(arch)-rpms`, à activer avec `sudo subscription-manager repos --enable`
EPEL (Rocky, AlmaLinux)	`sudo dnf install epel-release`

Écrire un fichier .repo à la main

/etc/yum.repos.d/interne.repo

```ini
[interne]
name=Dépôt interne de l'entreprise
baseurl=https://repo.exemple.lan/el9/$basearch/
enabled=1
gpgcheck=1
gpgkey=https://repo.exemple.lan/RPM-GPG-KEY-interne
```

> **GPGCHECK=1, TOUJOURS**
>
> Ne désactivez jamais la vérification des signatures (`gpgcheck=0`) sur un dépôt de production : sans elle, rien ne garantit que les paquets n'ont pas été modifiés entre le dépôt et votre serveur.

FICHE 16 · DÉPÔTS

16 `dnf group` — Installer un ensemble de paquets

OBJECTIF	PRIVILÈGES	NIVEAU
Déployer un environnement complet en une commande	root (sudo) pour installer	**Pratique**

Un **groupe** rassemble des paquets qui servent un même but : outils de développement, administration, sécurité… Les **groupes d'environnement** vont plus loin et décrivent un système entier, comme « Server with GUI » ou « Minimal Install ».

```bash
$ dnf group list
Available Environment Groups:
   Server with GUI
   Server
   Minimal Install
   Workstation
Installed Groups:
   Container Management
Available Groups:
   Development Tools
   Headless Management
   System Tools
$ dnf group info "Development Tools"
Group: Development Tools
 Description: A basic development environment.
 Mandatory Packages:
   autoconf
   automake
   gcc
   …
```

```bash
$ sudo dnf group install "Development Tools"
# Avec les paquets optionnels du groupe
$ sudo dnf group install --with-optional "Development Tools"
# Syntaxe abrégée : l'identifiant du groupe précédé de @
$ sudo dnf install @development
```

> **NOMS OU IDENTIFIANTS ?**
>
> Les noms affichés peuvent être traduits selon la langue du système, pas les identifiants. `dnf group list -v` affiche les identifiants entre parenthèses : `development` sur RHEL et dérivés, `development-tools` sur Fedora. Dans un script, utilisez toujours l'identifiant.

FICHE 17 · DÉPÔTS

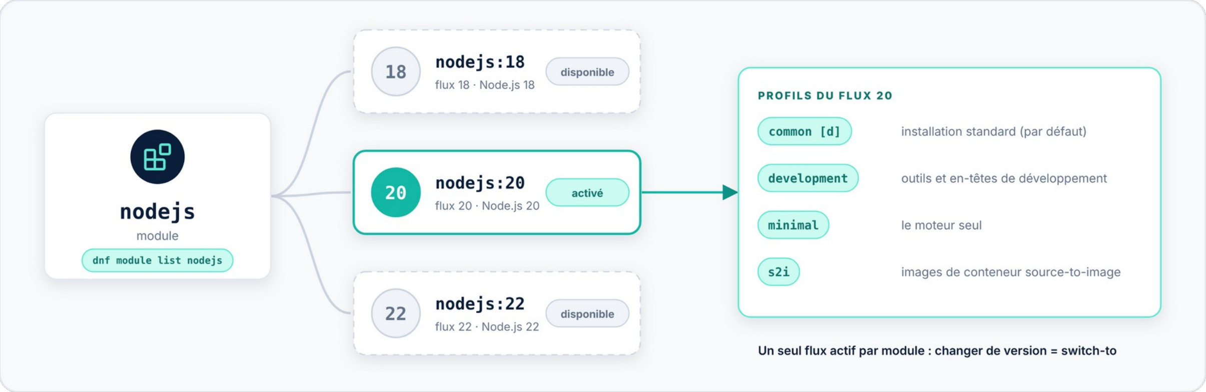
17 `dnf module` — Choisir la version d'une application

OBJECTIF
Installer Node.js, PHP ou PostgreSQL dans la version voulue

PRIVILÈGES
root (sudo) pour activer

NIVEAU
Avancé

Sur RHEL 8 et 9 et leurs dérivés, le dépôt AppStream propose plusieurs versions de certaines applications sous forme de **modules**. Chaque module offre des **flux** (*streams*, une version majeure) et des **profils** (un ensemble de paquets adapté à un usage).


nodejs
module
dnf module list nodejs
18
nodejs:18
flux 18 · Node.js 18
disponible
20
nodejs:20
flux 20 · Node.js 20
activé
22
nodejs:22
flux 22 · Node.js 22
disponible
PROFILS DU FLUX 20
common [d]
installation standard (par défaut)
development
outils et en-têtes de développement
minimal
le moteur seul
s2i
images de conteneur source-to-image

Un seul flux actif par module : changer de version = switch-to

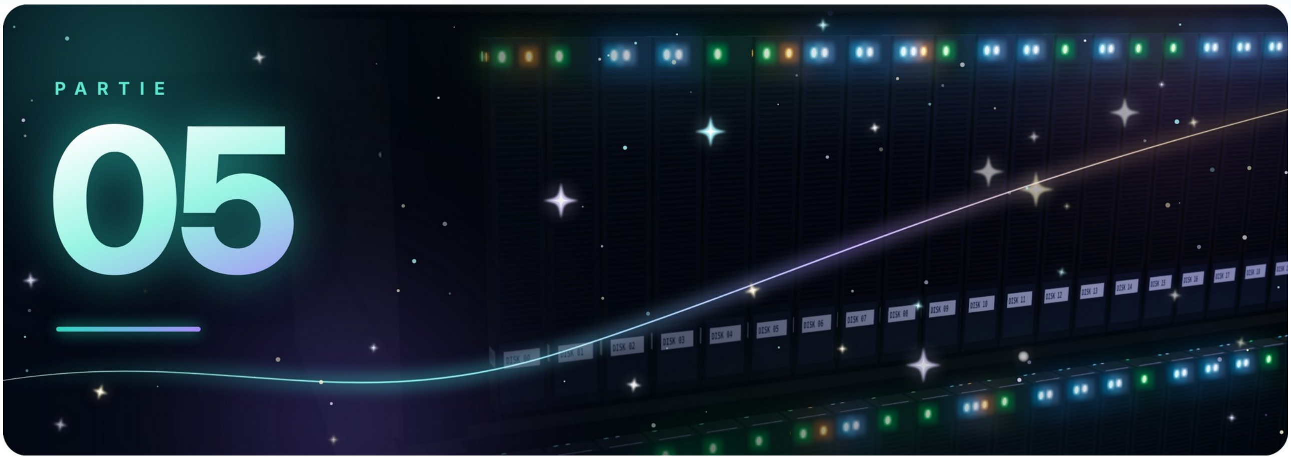
Figure 4 — *Un module, ses flux et ses profils : un seul flux peut être actif à la fois.*

```bash
$ dnf module list nodejs
Rocky Linux 9 - AppStream
Name      Stream    Profiles                                   Summary
nodejs    18        common [d], development, minimal, s2i      Javascript runtime
nodejs    20        common [d], development, minimal, s2i      Javascript runtime
nodejs    22        common [d], development, minimal, s2i      Javascript runtime

Hint: [d]efault, [e]nabled, [x]disabled, [i]nstalled
$ sudo dnf module enable nodejs:20
$ sudo dnf install nodejs
# Changer de version : basculer vers un autre flux
$ sudo dnf module switch-to nodejs:22
# Revenir à l'état initial du module
$ sudo dnf module reset nodejs
```

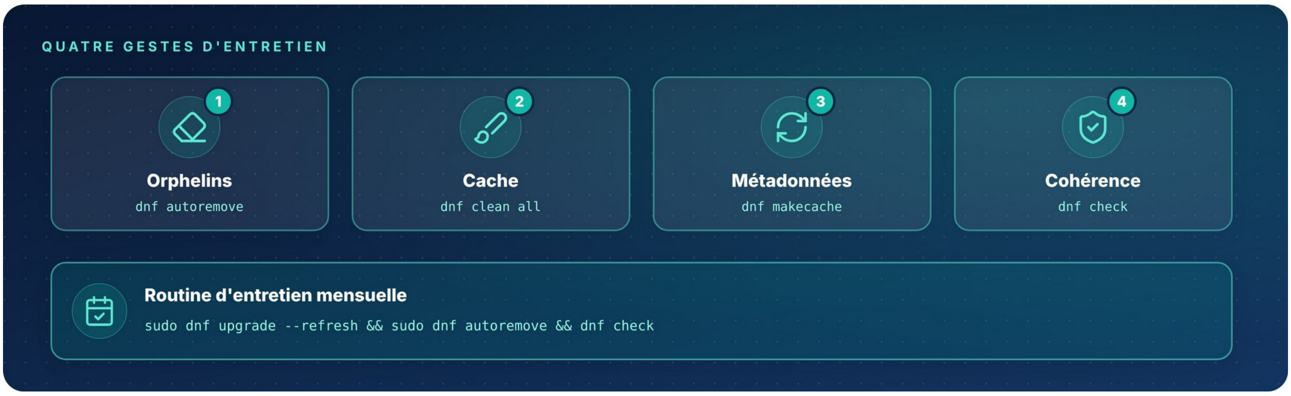
UNE TECHNOLOGIE EN FIN DE VIE

Fedora a abandonné les modules à partir de la version 39, et RHEL 10 n'en propose plus : les versions alternatives y sont livrées comme des paquets ordinaires (sur Fedora, par exemple, `nodejs20` et `nodejs22`). Sur RHEL 8 et 9, les modules restent la méthode officielle.



Entretien et vérification

Un système sain s’entretient : supprimer les dépendances devenues inutiles, garder un cache propre et vérifier régulièrement la cohérence de la base RPM et des fichiers installés.



QUATRE GESTES D'ENTRETIEN

1. **Orphelins** `dnf autoremove`
2. **Cache** `dnf clean all`
3. **Métadonnées** `dnf makecache`
4. **Cohérence** `dnf check`

Routine d'entretien mensuelle

```
sudo dnf upgrade --refresh && sudo dnf autoremove && dnf check
```

DANS CETTE PARTIE

18	`dnf autoremove`	Supprimer les dépendances orphelines
19	`dnf clean`	Vider et reconstruire le cache
20	`dnf check`	Vérifier la cohérence de la base RPM

FICHE 18 · ENTRETIEN

18 `dnf autoremove` — Supprimer les dépendances orphelines

OBJECTIF	PRIVILÈGES	NIVEAU
Récupérer de l'espace, réduire la surface d'attaque	root (sudo)	**Pratique**

Lorsqu'un paquet est supprimé, ses dépendances peuvent rester sur le disque sans plus servir à rien. `dnf autoremove` supprime tous les paquets installés **comme dépendance** dont plus aucun paquet n'a besoin.

```bash
# Prévisualiser ce qui serait supprimé
$ dnf repoquery --unneeded
$ sudo dnf autoremove
Removing:
 gd                  x86_64     2.3.2-3.el9            @appstream       406 k
 libXpm              x86_64     3.5.13-10.el9          @appstream       114 k
 libwebp             x86_64     1.2.0-8.el9            @appstream       788 k

Transaction Summary
================================================================================
Remove  3 Packages

Freed space: 1.3 M
Is this ok [y/N]:
```

Protéger un paquet indispensable

Un paquet installé comme dépendance, mais que vous utilisez directement, risque d'être emporté. Marquez-le comme installé par l'utilisateur :

```bash
$ sudo dnf mark install gd          # DNF 4
$ sudo dnf mark user gd             # DNF5
```

Faire le ménage dans les anciens noyaux

DNF conserve plusieurs noyaux (`installonly_limit=3` par défaut) pour permettre un retour en arrière au démarrage. Pour n'en garder que deux :

```bash
$ sudo dnf remove --oldinstallonly --setopt installonly_limit=2 kernel
```

FICHE 19 · ENTRETIEN

19 `dnf clean` — Vider et reconstruire le cache

OBJECTIF	PRIVILÈGES	NIVEAU
Libérer de l’espace, forcer des métadonnées fraîches	root (sudo)	**Pratique**

DNF garde en cache les métadonnées des dépôts et, selon la configuration, les paquets téléchargés. Un cache corrompu ou périmé explique bien des erreurs étranges : `dnf clean` le vide, `dnf makecache` le reconstruit.

```bash
$ du -sh /var/cache/dnf
412M    /var/cache/dnf
$ sudo dnf clean all
68 files removed
$ sudo dnf makecache
Rocky Linux 9 - BaseOS                          3.1 MB/s | 2.4 MB     00:00
Rocky Linux 9 - AppStream                       6.8 MB/s | 8.9 MB     00:01
Rocky Linux 9 - Extras                           38 kB/s |  16 kB     00:00
Metadata cache created.
```

COMMANDE	CE QUI EST SUPPRIMÉ
`dnf clean all`	Tout : métadonnées, paquets et cache dérivé
`dnf clean metadata`	Métadonnées des dépôts — le plus utile après un changement de dépôt
`dnf clean packages`	Paquets RPM conservés dans le cache
`dnf clean dbcache`	Fichiers de cache calculés à partir des métadonnées
`dnf clean expire-cache`	Rien : les métadonnées sont seulement marquées comme expirées

> **QUAND NETTOYER ?**
> Après l’ajout ou la modification d’un dépôt, en cas d’erreur « Failed to download metadata » ou de somme de contrôle, ou quand `/var` manque d’espace. Inutile avant chaque mise à jour : l’option `--refresh` suffit.

FICHE 20 · ENTRETIEN

20 `dnf check` — Vérifier la cohérence de la base RPM

OBJECTIF	PRIVILÈGES	NIVEAU
Détecter dépendances cassées et doublons	Aucun	Avancé

Une mise à jour interrompue, un `rpm --nodeps` malheureux ou un paquet installé hors DNF peuvent rendre la base RPM incohérente. `dnf check` la contrôle : dépendances manquantes, paquets en double, paquets rendus obsolètes. Aucune sortie signifie aucun problème.

```bash
$ dnf check
$ echo $?
0
# Après une mise à jour interrompue, par exemple :
$ dnf check
systemd-252-51.el9.x86_64 is a duplicate with systemd-252-46.el9.x86_64
python3-foo-1.2-3.el9.noarch has missing requires of python3-bar >= 2.0
# Supprimer les anciennes versions en double
$ sudo dnf remove --duplicates
```

Aller plus loin : vérifier les fichiers avec rpm -V

`dnf check` contrôle la base de paquets ; `rpm -V` contrôle les **fichiers** installés en les comparant aux informations du paquet. Chaque caractère signale un écart :

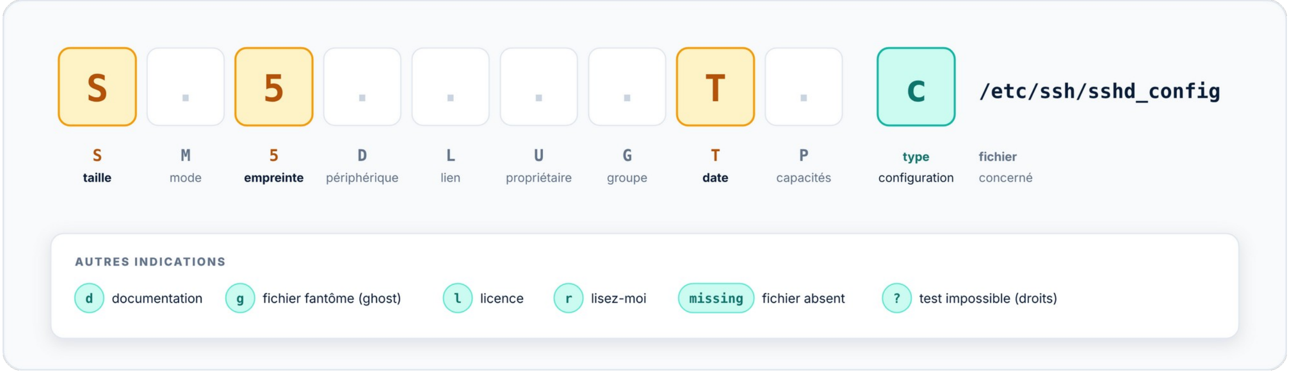

S . 5 . . . . T . c /etc/ssh/sshd_config
S taille
M mode
5 empreinte
D périphérique
L lien
U propriétaire
G groupe
T date
P capacités
type configuration
fichier concerné
AUTRES INDICATIONS
d documentation
g fichier fantôme (ghost)
l licence
r lisez-moi
missing fichier absent
? test impossible (droits)

Figure 5 — *Lecture d'une ligne de rpm -V : un point signifie « conforme ».*

```bash
$ rpm -V openssh-server
S.5....T.  c /etc/ssh/sshd_config
# Tous les paquets (plusieurs minutes), sans les fichiers de configuration
$ sudo rpm -Va | grep -v ' c /'
```

> **COMMENT INTERPRÉTER**
>
> Un fichier de configuration modifié (`c`) est normal : c'est votre configuration. En revanche, un binaire de `/usr/bin` ou `/usr/sbin` dont l'empreinte (`5`) a changé sans mise à jour doit être examiné de près.

ÉVOLUTION

DNF5, la nouvelle génération

Depuis Fedora 41 (octobre 2024), la commande `dnf` lance DNF5, une réécriture complète en C++ : plus rapide, plus légère, avec une seule bibliothèque partagée par l'outil en ligne de commande et les interfaces graphiques. RHEL 10 et ses dérivés restent, eux, sur DNF 4.

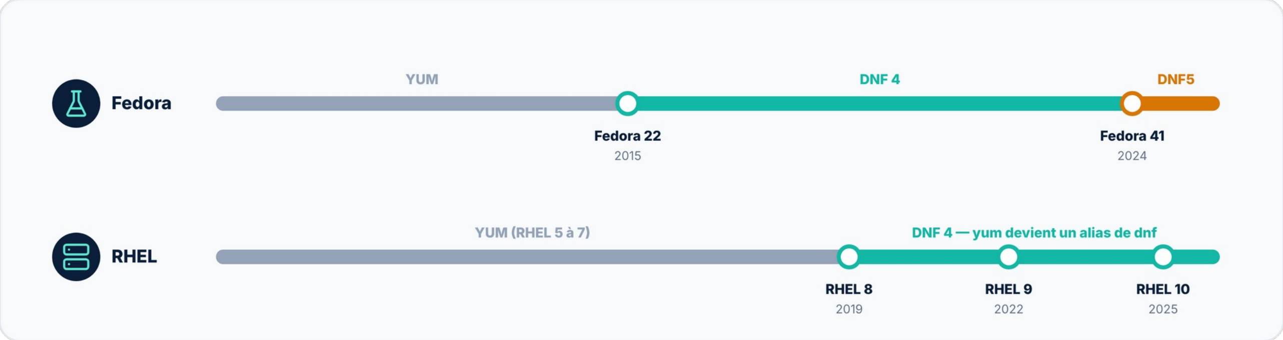

Fedora
YUM
DNF 4
DNF5
Fedora 22
2015
Fedora 41
2024
RHEL
YUM (RHEL 5 à 7)
DNF 4 — yum devient un alias de dnf
RHEL 8
2019
RHEL 9
2022
RHEL 10
2025

Figure 6 — *De YUM à DNF5 : trois générations de gestionnaires de paquets.*

Bonne nouvelle : la grande majorité des commandes de ce guide fonctionnent à l'identique avec DNF5. Les différences portent surtout sur la configuration des dépôts, l'affichage et quelques options.

Les commandes en pratique

```bash
$ sudo dnf5 install nginx
$ sudo dnf5 remove nginx
$ sudo dnf5 upgrade --refresh
$ dnf5 check-upgrade
$ dnf5 list --installed
$ dnf5 repo list --all
$ sudo dnf5 config-manager addrepo --from-repofile=https://exemple.org/app.repo
$ sudo dnf5 config-manager setopt epel.enabled=1
$ dnf5 history list
$ sudo dnf5 history undo 42
$ sudo dnf5 clean all
```

Ce qui change au quotidien

SUJET	DNF 4	DNF5
Implémentation	Python, libdnf	C++, libdnf5
Distributions	RHEL 8 à 10, Rocky, Alma, CentOS Stream	Fedora 41 et suivantes
Vérifier les mises à jour	`dnf check-update`	`dnf check-upgrade` (ancien nom accepté)
Lister les installés	`list installed` ou `--installed`	`list --installed` seulement
Lister les dépôts	`dnf repolist`	`dnf repo list` (ou `repolist`)
Ajouter un dépôt	`config-manager --add-repo`	`config-manager addrepo`
Activer un dépôt	`config-manager --set-enabled`	`config-manager setopt`
Marquer un paquet	`dnf mark install pkg`	`dnf mark user pkg`
Modules	Support complet	Support partiel (enable, disable, reset, list)
Cache	`/var/cache/dnf`	`/var/cache/libdnf5`
Journal	`/var/log/dnf.log`	`/var/log/dnf5.log`

Bon à savoir avant de migrer

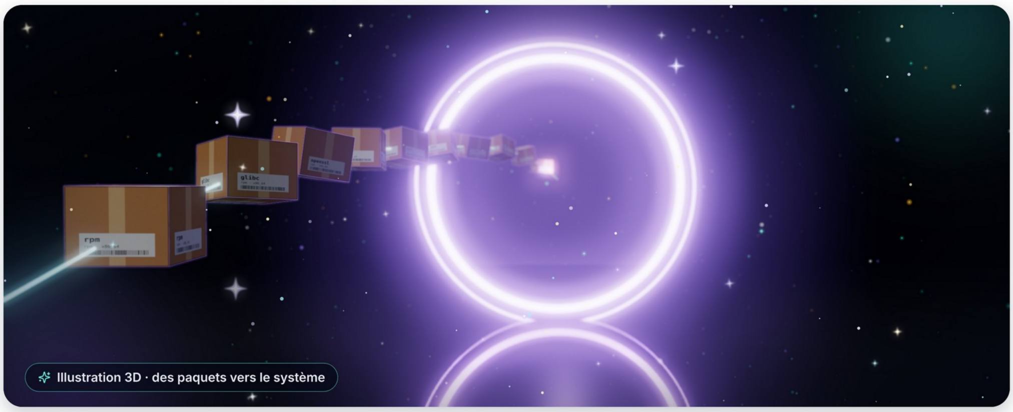
- Sur Fedora 41 et suivantes, `dnf` **est** DNF5 : la commande `dnf5` n'est qu'un autre nom. L'ancien DNF reste disponible sous le nom `dnf4` pour la transition ;
- Les **scripts qui analysent la sortie** de DNF (colonnes, libellés) doivent être revérifiés : l'affichage a changé ;
- Les **greffons** de DNF 4 (`dnf-plugins-core`) ne fonctionnent pas avec DNF5, qui a ses propres extensions (`dnf5-plugins`) ;
- Les **listes de fichiers** des dépôts ne sont plus téléchargées par défaut : DNF5 les récupère à la demande, par exemple pour `dnf provides /chemin/fichier` ;
- Préférez dès aujourd'hui les **formes compatibles** avec les deux versions : `dnf list --installed` plutôt que `list installed`, `dnf upgrade` plutôt que `update`.

> **UNE SEULE HABITUDE À PRENDRE**
>
> Écrivez vos procédures avec la syntaxe commune aux deux versions et ne réservez les formes propres à DNF5 qu'à la configuration des dépôts (`config-manager addrepo` et `setopt`). Vos runbooks resteront valables sur RHEL comme sur Fedora.

POUR CONCLURE

Conclusion

DNF est bien plus qu'un installateur : c'est un outil de **gestion du changement**. Chaque transaction est calculée, présentée, signée, journalisée et, dans la plupart des cas, réversible. Utilisé avec méthode, il rend la maintenance d'un parc Red Hat prévisible.


rpm
glibc
Illustration 3D · des paquets vers le système

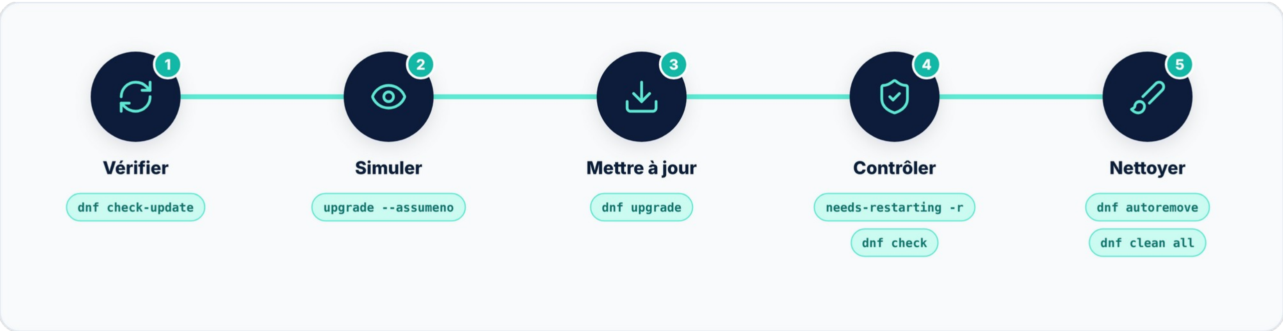

1
Vérifier
dnf check-update
2
Simuler
upgrade --assumeno
3
Mettre à jour
dnf upgrade
4
Contrôler
needs-restarting -r
dnf check
5
Nettoyer
dnf autoremove
dnf clean all

Figure 7 — *La routine de maintenance en cinq temps.*

Les réflexes à retenir

- **Lire le résumé de transaction** avant de répondre `y`, en particulier les sections « Removing » ;
- **`dnf check-update`** avant chaque mise à jour, et son code 100 dans vos scripts ;
- **`dnf history`** pour comprendre, **`history undo`** ou **`downgrade`** pour revenir en arrière ;
- **Jamais** **`--nogpgcheck`** ni `gpgcheck=0` en production ;
- **Les identifiants** de groupes et de dépôts plutôt que les noms affichés dans les scripts ;
- **La syntaxe à tirets** (`--installed`, `--available`) : elle fonctionne aussi avec DNF5.

> **POUR ALLER PLUS LOIN**
> Automatisez l'application des correctifs avec `dnf-automatic` (minuteur systemd `dnf-automatic.timer`) ; pour passer d'une version de Fedora à la suivante, utilisez `dnf system-upgrade`.

RESSOURCES

Annexes

Ces annexes rassemblent les informations à garder sous la main : aide-mémoire par besoin, équivalences avec apt, fichiers de DNF, réglages utiles, messages d’erreur et glossaire.

A. Aide-mémoire : « Je veux… »

JE VEUX…	COMMANDE
Installer nginx	`sudo dnf install nginx`
Installer un RPM téléchargé	`sudo dnf install ./paquet.rpm`
Désinstaller un paquet	`sudo dnf remove nginx`
Voir les mises à jour disponibles	`dnf check-update`
Tout mettre à jour	`sudo dnf upgrade --refresh`
Seulement les correctifs de sécurité	`sudo dnf upgrade --security`
Savoir s’il faut redémarrer	`sudo dnf needs-restarting -r`
Trouver le paquet d’une commande	`dnf provides '*/bin/dig'`
Annuler la dernière transaction	`sudo dnf history undo last`
Revenir à la version précédente	`sudo dnf downgrade nginx`
Activer CRB durablement	`sudo dnf config-manager --set-enabled crb`
Installer les outils de développement	`sudo dnf group install "Development Tools"`
Libérer de l’espace	`sudo dnf autoremove && sudo dnf clean all`

B. Équivalences apt ↔ dnf

ACTION	DEBIAN / UBUNTU	FAMILLE RED HAT
Rafraîchir les métadonnées	`apt update`	`dnf makecache`
Lister les mises à jour	`apt list --upgradable`	`dnf check-update`
Tout mettre à jour	`apt upgrade`	`dnf upgrade`
Installer	`apt install nginx`	`dnf install nginx`
Supprimer	`apt remove nginx`	`dnf remove nginx`
Supprimer les orphelins	`apt autoremove`	`dnf autoremove`
Rechercher	`apt search motclé`	`dnf search motclé`
Détails d'un paquet	`apt show nginx`	`dnf info nginx`
Paquet fournissant un fichier	`apt-file search fichier`	`dnf provides fichier`
Historique	`/var/log/apt/history.log`	`dnf history`
Vider le cache	`apt clean`	`dnf clean all`
Outil bas niveau	`dpkg` (paquets `.deb`)	`rpm` (paquets `.rpm`)

C. Fichiers et répertoires de DNF

EMPLACEMENT	RÔLE
`/etc/dnf/dnf.conf`	Configuration principale (section [main])
`/etc/yum.repos.d/*.repo`	Définition des dépôts
`/etc/dnf/protected.d/`	Paquets protégés contre la suppression
`/etc/pki/rpm-gpg/`	Clés GPG publiques des dépôts
`/var/cache/dnf/`	Cache des métadonnées et des paquets (DNF 4)
`/var/lib/dnf/history.sqlite`	Historique des transactions (DNF 4)
`/var/log/dnf.log, dnf.rpm.log`	Journaux de DNF 4 et des opérations RPM
`/var/cache/libdnf5/`	Cache de DNF5
`/var/log/dnf5.log`	Journal de DNF5
`/var/lib/rpm/`	Base RPM des paquets installés (lien vers /usr/lib/sysimage/rpm sur Fedora)

D. Réglages utiles de dnf.conf

`/etc/dnf/dnf.conf`

```
[main]
gpgcheck=True
installonly_limit=3
clean_requirements_on_remove=True
best=True
skip_if_unavailable=False
max_parallel_downloads=10
keepcache=False
```

RÉGLAGE	EFFET
`gpgcheck`	Vérifier la signature de chaque paquet (à laisser activé)
`installonly_limit`	Nombre de noyaux conservés (3 par défaut)
`clean_requirements_on_remove`	Supprimer les dépendances inutiles avec le paquet
`best`	Exiger la version la plus récente, ou échouer
`max_parallel_downloads`	Téléchargements simultanés (3 par défaut, 20 au maximum)
`fastestmirror`	Choisir le miroir le plus rapide (à tester : gain variable)
`keepcache`	Conserver les RPM téléchargés (utile hors ligne ou pour undo)
`defaultyes`	Proposer « oui » par défaut à la confirmation
`exclude`	Paquets ignorés par toutes les commandes

E. Messages d’erreur courants

MESSAGE	CAUSE ET SOLUTION
`Error: Unable to find a match: xyz`	Nom erroné ou dépôt désactivé (EPEL, CRB) : `dnf search`, `dnf provides`, `dnf repolist --all`
`Failed to download metadata for repo 'xyz'`	Réseau, proxy ou miroir indisponible : vérifier l’URL, `dnf clean metadata`, ou désactiver le dépôt
`GPG check FAILED` / `Public key … is not installed`	Importer la clé de l’éditeur (`rpm --import`) après vérification — jamais `--nogpgcheck`
`Waiting for process with pid 1234 to finish.`	Un autre DNF (ou PackageKit) travaille : patienter, puis vérifier le processus
`… conflicts with file from package …`	Deux paquets livrent le même fichier : choisir l’un d’eux ou, après analyse, `--allowerasing`
`nothing provides … needed by …`	Dépendance absente des dépôts actifs : activer CRB ou EPEL, ou utiliser `--skip-broken`
`… removing the following protected packages`	Paquet vital protégé : revoir la transaction plutôt que forcer
`All matches were filtered out by modular filtering`	Le paquet appartient à un flux non activé : `dnf module enable` ou `reset`

F. Glossaire

TERME	DÉFINITION
RPM	Format de paquet et outil bas niveau de la famille Red Hat
Dépôt	Serveur ou répertoire qui publie des paquets et leurs métadonnées
Métadonnées	Index d’un dépôt (repodata) : liste des paquets, dépendances, avis de sécurité
Transaction	Ensemble cohérent d’opérations calculé et appliqué en une fois par DNF
NEVRA	Nom, époque, version, release, architecture : l’identité complète d’un paquet
BaseOS / AppStream	Dépôts principaux de RHEL : système de base et applications
EPEL / CRB	Paquets supplémentaires de Fedora pour RHEL / outils de développement
Groupe	Ensemble de paquets installables en une commande
Module, flux, profil	Application, version majeure et ensemble de paquets (RHEL 8 et 9)
installonly	Paquets installés côte à côte au lieu d’être remplacés (noyau)

G. Sources et références

- Anusha Saive, « 20 commandes DNF que tout administrateur Linux devrait connaître », Tecmint (tecmint.com), mis à jour le 1er septembre 2026. Article dont ce guide reprend le plan et qu'il corrige et enrichit.
- Pages de manuel : `dnf(8)`, `dnf.conf(5)`, `dnf5(8)`, `rpm(8)` ; documentation de DNF (dnf.readthedocs.io) et de DNF5 (dnf5.readthedocs.io).
- Documentation Red Hat Enterprise Linux 9 : « Managing software with the DNF tool » ; documentation Fedora sur le passage à DNF5.

DOCUMENT RÉDIGÉ ET MIS EN FORME PAR

Ngouye GNING

Version 1.0 · Octobre 2026